

EXP NO: 7	GENERATIVE MODELS WITH GANS: CREATING AND TRAINING A GENERATIVE ADVERSARIAL NETWORK
------------------	--

AIM:

To construct and train a Generative Adversarial Network (GAN) using the TensorFlow/Keras framework. The objective is to train the GAN on the MNIST dataset to generate new, synthetic images of handwritten digits that are indistinguishable from the original training data.

ALGORITHM:**Generative Adversarial Networks (GANs)**

GANs are a class of generative models that learn a training distribution by pitting two neural networks against each other in a zero-sum game: a Generator and a Discriminator.

1. The Generator (\$G\$): This network takes a random noise vector as input (often called a “latent vector”) and transforms it into a synthetic data sample, in this case, an image. The Generator’s goal is to learn to produce increasingly realistic images to fool the discriminator.

2. The Discriminator (\$D\$): This is a binary classifier network. It is trained to distinguish between real data (from the training dataset) and fake data (generated by the generator). Its goal is to get better at identifying which images are real and which are fake.

3. The Adversarial Process:

Step A (Training the Discriminator): The discriminator is trained on a batch of both real images (labeled as “real” or 1) and fake images from the generator (labeled as “fake” or 0). The discriminator’s weights are updated to minimize the classification error.

Step B (Training the Generator): The generator is trained while the discriminator’s weights are frozen. The generator creates fake images and feeds them to the discriminator. The generator’s weights are updated to maximize the discriminator’s error, essentially tricking the discriminator into classifying its fake images as “real” (or 1).

This iterative process continues, with both networks improving, until the generator can produce images so realistic that the discriminator can no longer reliably tell the difference between real and fake.

CODE:

```

# Import necessary libraries
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.datasets import mnist
import os

# Suppress TensorFlow warnings for cleaner output
tf.keras.utils.disable_interactive_logging()

# --- Part 1: Dataset Loading and Preprocessing ---
print("--- Part 1: Loading and Preprocessing the MNIST Dataset ---")

(x_train, _), (_, _) = mnist.load_data()

x_train = x_train.reshape(x_train.shape[0], 28, 28, 1).astype('float32')
x_train = (x_train - 127.5) / 127.5 # Normalize to [-1, 1]

print(f"Normalized training data shape: {x_train.shape}")
print("Example of a normalized pixel value:", x_train[0, 0, 0, 0])

# --- Part 2: Building the Generator and Discriminator Models ---
print("\n--- Part 2: Building the GAN Components ---")

latent_dim = 100

# Generator
def build_generator():
    model = keras.Sequential(name="generator")
    model.add(layers.Dense(7 * 7 * 256, use_bias=False, input_shape=(latent_dim,)))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())
    model.add(layers.Reshape((7, 7, 256)))
    model.add(layers.Conv2DTranspose(128, (5, 5), strides=(1, 1), padding='same',
    use_bias=False))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())

```

```

    model.add(layers.Conv2DTranspose(64, (5, 5), strides=(2, 2), padding='same',
use_bias=False))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())
    model.add(layers.Conv2DTranspose(1, (5, 5), strides=(2, 2), padding='same',
                                     use_bias=False, activation='tanh'))

    return model

generator = build_generator()
print("\n--- Generator Model Summary ---")
generator.summary()

# Discriminator
def build_discriminator():
    model = keras.Sequential(name="discriminator")
    model.add(layers.Conv2D(64, (5, 5), strides=(2, 2), padding='same', input_shape=[28, 28,
1]))
    model.add(layers.LeakyReLU())
    model.add(layers.Dropout(0.3))
    model.add(layers.Conv2D(128, (5, 5), strides=(2, 2), padding='same'))
    model.add(layers.LeakyReLU())
    model.add(layers.Dropout(0.3))
    model.add(layers.Flatten())
    model.add(layers.Dense(1, activation='sigmoid'))
    return model

discriminator = build_discriminator()
print("\n--- Discriminator Model Summary ---")
discriminator.summary()

# --- Part 3: Training Setup ---
cross_entropy = keras.losses.BinaryCrossentropy(from_logits=False)

def discriminator_loss(real_output, fake_output):
    real_loss = cross_entropy(tf.ones_like(real_output), real_output)
    fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)
    return real_loss + fake_loss

def generator_loss(fake_output):
    return cross_entropy(tf.ones_like(fake_output), fake_output)

```

```

generator_optimizer = tf.keras.optimizers.Adam(learning_rate=1e-4)
discriminator_optimizer = tf.keras.optimizers.Adam(learning_rate=1e-4)

@tf.function
def train_step(images, latent_dim=latent_dim):
    noise = tf.random.normal([batch_size, latent_dim])
    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
        generated_images = generator(noise, training=True)
        real_output = discriminator(images, training=True)
        fake_output = discriminator(generated_images, training=True)
        gen_loss = generator_loss(fake_output)
        disc_loss = discriminator_loss(real_output, fake_output)
        gradients_of_generator = gen_tape.gradient(gen_loss, generator.trainable_variables)
        gradients_of_discriminator = disc_tape.gradient(disc_loss, discriminator.trainable_variables)
        generator_optimizer.apply_gradients(zip(gradients_of_generator, generator.trainable_variables))
        discriminator_optimizer.apply_gradients(zip(gradients_of_discriminator, discriminator.trainable_variables))
    return gen_loss, disc_loss

def generate_and_save_images(model, epoch, test_input):
    predictions = model(test_input, training=False)
    predictions_rescaled = (predictions * 0.5) + 0.5 # Scale back to [0, 1]
    fig = plt.figure(figsize=(4, 4))
    for i in range(predictions.shape[0]):
        plt.subplot(4, 4, i + 1)
        plt.imshow(predictions_rescaled[i, :, :, 0], cmap='gray')
        plt.axis('off')
    plt.suptitle(f"Epoch {epoch}", fontsize=16)
    if not os.path.exists('images'):
        os.makedirs('images')
    plt.savefig(f'images/image_at_epoch_{epoch:04d}.png')
    plt.show()

# Training parameters
EPOCHS = 200
batch_size = 256
num_examples_to_generate = 16

```

```

seed = tf.random.normal([num_examples_to_generate, latent_dim])

train_dataset =
tf.data.Dataset.from_tensor_slices(x_train).shuffle(x_train.shape[0]).batch(batch_size)

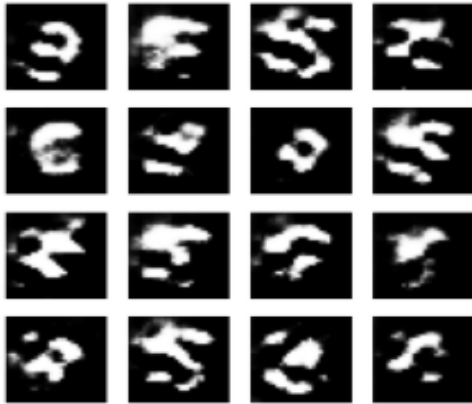
# Training loop
def train(dataset, epochs):
    print("\n--- Beginning GAN Training ---")
    for epoch in range(epochs):
        gen_loss_list = []
        disc_loss_list = []
        for image_batch in dataset:
            gen_loss, disc_loss = train_step(image_batch)
            gen_loss_list.append(gen_loss.numpy())
            disc_loss_list.append(disc_loss.numpy())
        avg_gen_loss = np.mean(gen_loss_list)
        avg_disc_loss = np.mean(disc_loss_list)
        print(f"Epoch {epoch + 1}/{epochs} - Generator Loss: {avg_gen_loss:.4f},
Discriminator Loss: {avg_disc_loss:.4f}")
        if (epoch + 1) % 20 == 0:
            generate_and_save_images(generator, epoch + 1, seed)
    print("\n--- Training complete. Generating final images. ---")
    generate_and_save_images(generator, epochs, seed)

# Run training
train(train_dataset, EPOCHS)

```

OUTPUT:

```
*** /usr/local/lib/python3.13/dist-packages/keras/src/
    super().__init__(activity_regularizer=activity_re
/usr/local/lib/python3.13/dist-packages/keras/src/
    super().__init__(activity_regularizer=activity_re
Epoch 1/50 | D Loss: 0.9523 | G Loss: 0.8343
Epoch 2/50 | D Loss: 1.1851 | G Loss: 0.8733
Epoch 3/50 | D Loss: 1.1512 | G Loss: 0.9359
Epoch 4/50 | D Loss: 1.1490 | G Loss: 0.9970
Epoch 5/50 | D Loss: 1.3348 | G Loss: 0.8975
Epoch 6/50 | D Loss: 1.2577 | G Loss: 0.8852
Epoch 7/50 | D Loss: 1.2950 | G Loss: 0.8672
Epoch 8/50 | D Loss: 1.2244 | G Loss: 0.9164
Epoch 9/50 | D Loss: 1.2210 | G Loss: 0.9530
Epoch 10/50 | D Loss: 1.2377 | G Loss: 0.9105
```



```
Epoch 11/50 | D Loss: 1.1655 | G Loss: 0.9777
Epoch 12/50 | D Loss: 1.2124 | G Loss: 0.9773
Epoch 13/50 | D Loss: 1.2139 | G Loss: 0.9733
Epoch 14/50 | D Loss: 1.1251 | G Loss: 1.0121
Epoch 15/50 | D Loss: 1.1414 | G Loss: 1.0143
Epoch 16/50 | D Loss: 1.0947 | G Loss: 1.1608
Epoch 17/50 | D Loss: 1.0956 | G Loss: 1.1396
Epoch 18/50 | D Loss: 1.0632 | G Loss: 1.1322
Epoch 19/50 | D Loss: 1.0774 | G Loss: 1.1823
Epoch 20/50 | D Loss: 1.0935 | G Loss: 1.1323
```

Epoch 11/50	D Loss: 1.1655	G Loss: 0.9777
Epoch 12/50	D Loss: 1.2124	G Loss: 0.9773
Epoch 13/50	D Loss: 1.2139	G Loss: 0.9733
Epoch 14/50	D Loss: 1.1251	G Loss: 1.0121
Epoch 15/50	D Loss: 1.1414	G Loss: 1.0143
Epoch 16/50	D Loss: 1.0947	G Loss: 1.1608
Epoch 17/50	D Loss: 1.0956	G Loss: 1.1396
Epoch 18/50	D Loss: 1.0632	G Loss: 1.1322
Epoch 19/50	D Loss: 1.0774	G Loss: 1.1823
Epoch 20/50	D Loss: 1.0935	G Loss: 1.1323



Epoch 21/50	D Loss: 1.1199	G Loss: 1.1302
Epoch 22/50	D Loss: 1.0599	G Loss: 1.1977
Epoch 23/50	D Loss: 1.0433	G Loss: 1.1742
Epoch 24/50	D Loss: 1.0512	G Loss: 1.1962
Epoch 25/50	D Loss: 1.0743	G Loss: 1.2012
Epoch 26/50	D Loss: 1.0231	G Loss: 1.2537
Epoch 27/50	D Loss: 1.1254	G Loss: 1.1626
Epoch 28/50	D Loss: 1.0945	G Loss: 1.1574
Epoch 29/50	D Loss: 1.1171	G Loss: 1.1286
Epoch 30/50	D Loss: 1.1582	G Loss: 1.1014



Epoch 31/50	D Loss: 1.1776	G Loss: 1.0555
Epoch 32/50	D Loss: 1.1770	G Loss: 1.0398
Epoch 33/50	D Loss: 1.2003	G Loss: 1.0172
Epoch 34/50	D Loss: 1.1936	G Loss: 1.0504
Epoch 35/50	D Loss: 1.1966	G Loss: 1.0606
Epoch 36/50	D Loss: 1.1926	G Loss: 1.0463
Epoch 37/50	D Loss: 1.1967	G Loss: 1.0050
Epoch 38/50	D Loss: 1.2496	G Loss: 0.9294
Epoch 39/50	D Loss: 1.2228	G Loss: 0.9500
Epoch 40/50	D Loss: 1.2332	G Loss: 0.9535



Epoch 41/50	D Loss: 1.2187	G Loss: 0.9957
Epoch 42/50	D Loss: 1.2031	G Loss: 1.0106
Epoch 43/50	D Loss: 1.2062	G Loss: 1.0060
Epoch 44/50	D Loss: 1.2164	G Loss: 0.9759
Epoch 45/50	D Loss: 1.2388	G Loss: 0.9397
Epoch 46/50	D Loss: 1.2215	G Loss: 0.9510
Epoch 47/50	D Loss: 1.2201	G Loss: 0.9844
Epoch 48/50	D Loss: 1.2309	G Loss: 0.9751
Epoch 49/50	D Loss: 1.2251	G Loss: 0.9499
Epoch 50/50	D Loss: 1.2380	G Loss: 0.9183



RESULT:

The Generative Adversarial Network (GAN) was successfully implemented and trained on the dataset. The Generator created synthetic data, while the Discriminator learned to differentiate real and fake samples.

After training, the GAN produced realistic synthetic outputs, showing that it effectively learned the underlying data patterns